

Manual Técnico de Move Smart

Este documento recopila información técnica detallada del sistema **Move Smart**. Está dirigido a desarrolladores, administradores y otros mantenedores.

1. Introducción

Propósito: describir la arquitectura, modelos de datos, procesos, dependencias y operación del software.

Alcance: cubre el backend (API REST con Node/Express), frontend (React/Vite) y la base de datos MySQL. No incluye sistemas externos o módulos futuros no implementados.

Referencias:

- [docs/arquitectura.md](#)
 - [docs/arquitectura_base_de_datos.md](#)
 - Esquema SQL en [backend/sql/schema.sql](#)
 - README principal del proyecto.
-

2. Requisitos del Sistema

Hardware

- CPU moderna de 2+ núcleos
- 2-4 GB RAM mínimos
- Espacio en disco 200 MB (más para datos)

Software

- Node.js 16+ (backend y herramientas)
- npm/yarn
- MySQL 8.0 (o compatible)
- Navegador reciente (Chrome, Firefox, Edge) para el frontend
- Git para control de versiones

Configuración

- Variables de entorno en el archivo [backend/.env](#) (HOST, USER, PASSWORD, DATABASE, JWT_SECRET, etc.)
- Base de datos inicializada mediante [backend/migrate-to-mysql.js](#) o ejecutando [schema.sql](#).

Seguridad

- JWT para autenticación (se espera `Authorization: Bearer <token>` en cabeceras).
 - Contraseñas almacenadas con hash SHA2/BCrypt (dependiendo de implementación).
-

3. Arquitectura del Software

3.1 Diseño general

El sistema adopta una **arquitectura monolítica modular** siguiendo el patrón **MVC**.

- **Backend** (Node.js + Express)
 - `config/db.js`: configura y exporta la conexión a MySQL.
 - `controllers/*Controller.js`: implementan la lógica de negocio.
 - `middlewares/`: manejadores de autenticación, errores y adjunción de la base.
 - `routes/*Routes.js`: definen los endpoints REST.
 - `index.js`: arranca el servidor y monta rutas.
 - `migrate-to-mysql.js`: convierte `db.json` de datos simulados en tablas MySQL.
- **Frontend** (React + Vite)
 - Estructura de componentes reutilizables (`components/`) para UI, gráficos, mapas, formularios.
 - Páginas (`pages/`) que representan vistas como Login, Dashboard, Mapa, CRUD de entidades.
 - `api/api.js`: funciones para invocar la API del backend con `fetch` o `axios`.

La comunicación se realiza mediante **API REST JSON**; el frontend consume rutas protegidas con JWT.

3.2 Componentes y librerías

- **Backend**: `express`, `mysql2/sequelize` (según configuración), `dotenv`, `jsonwebtoken`, `bcrypt`.
- **Frontend**: `react`, `react-dom`, `vite`, `react-router-dom`, `leaflet`, `recharts`, `axios`.

3.3 Diagrama de módulos

(Ver `docs/diagramas-casos-uso.yml` y `docs/diagramas-clases.yml` para representaciones en YAML; pueden convertirse a PlantUML o similar.)

Nota: los diagramas presentes en la carpeta `docs/mockups` muestran la interfaz y flujos.

4. Modelos de Datos

4.1 Conceptual

Las entidades principales son:

- Usuarios (con roles: analista, administrador, ciudadano).
- Rutas de transporte público.
- Paradas pertenecientes a rutas.
- Zonas críticas (regiones geográficas concentradas en accidentes o demanda).

4.2 Lógico y físico

La base MySQL contiene nueve tablas; algunas vistas auxiliares. El diccionario parcial:

Tabla	Campos clave	Descripción
-------	--------------	-------------

Tabla	Campos clave	Descripción
usuarios	id, email	credenciales y rol de usuarios
rutas	id, nombre	información de la ruta (origen, destino, distancia, estado, métricas)
paradas	id, ruta_id	lista de paradas con coordenadas y capacidad
zonas	id, nombre	zonas críticas geolocalizadas
...	...	...

El archivo `backend/sql/schema.sql` contiene la definición DDL completa con claves foráneas y restricciones.

Diccionario de datos (extracto)

- `usuarios.email`: VARCHAR(100), único.
- `rutas.distancia_km`: DECIMAL(10,2) en kilómetros.
- `paradas.latitud/longitud`: DECIMAL 10,8/11,8, coordenadas GPS.

Para detalles completos, consultar `docs/arquitectura_base_de_datos.md` que describe cada entidad con ejemplos JSON y mapeo desde el origen `db.json`.

5. Descripción del sistema

Move Smart es una plataforma **de simulación y gestión de rutas de transporte público**. Permite:

1. **Visualizar la red vial** en un mapa interactivo.
2. **Gestionar paradas, rutas y zonas críticas** mediante formularios CRUD.
3. **Simular optimizaciones** de rutas y calcular métricas de eficiencia/ocupación.
4. **Administrar usuarios** y sus roles para restringir funcionalidades.
5. **Generar reportes y aprobar cambios** (para analistas/administradores).

Desde la perspectiva de un usuario:

- **Ciudadano**: consulta rutas, paradas y mapas; no accede a ediciones.
- **Analista**: ejecuta simulaciones, visualiza métricas y propone cambios.
- **Administrador**: crea/edita datos, aprueba simulaciones y gestiona usuarios.

Atributos de calidad:

- **Escalabilidad**: modularidad permite migrar a microservicios.
- **Mantenibilidad**: código React dividido en componentes y clean-code en backend.
- **Seguridad**: autenticación JWT, roles y contraseñas hasheadas.
- **Rendimiento**: consultas indexadas, mapas renderizados en Leaflet.

6. Ejemplos y casos prácticos

1. **Instalación rápida**

```
cd backend
npm install
# configurar .env con credenciales
node migrate-to-mysql.js # importa datos de db.json
node index.js

cd ../frontend
npm install
npm run dev
```

2. Registro y login

- Crear usuario `POST /api/usuarios` con rol `ciudadano`.
- `POST /api/auth/login` recibe `{email,password}` y devuelve token.

3. Agregar una nueva ruta

- En Dashboard como administrador, ir a 'Rutas' -> 'Nueva ruta', completar formulario y enviar.
- Backend maneja `POST /api/rutas` que inserta en tabla `rutas`.

4. Simulación de optimización

- El analista accede a la sección de métricas, selecciona una ruta y pulsa "Simular".
- Se calcula eficiencia y se almacena en la misma tabla (`rutas.calificacion_eficiencia`).

5. Consultar paradas desde la API

```
GET /api/paradas?ruta_id=1
Authorization: Bearer <token>
```

Devuelve listado de paradas ordenadas por `orden`.

Estos casos ilustran interacciones típicas; el proyecto incluye más endpoints para CRUD completo y gestión de zonas.

7. Instalación y Configuración

(Ya descrita en Requisitos, repetir los pasos detallados con comandos, variables, precauciones de seguridad y ejemplos de `.env`.)

8. Módulos y funciones

En la carpeta `backend/controllers` cada módulo corresponde a una entidad (rutas, usuarios, paradas, zonas, etc.). Cada función exportada suele implementarse así:

```
// ejemplo: rutasController.js
const db = require('../config/db');

exports.crearRuta = async (req, res) => {
  const { nombre, origen, destino } = req.body;
  await db.query('INSERT INTO rutas (nombre, origen, destino) VALUES (?, ?, ?)',
    [nombre, origen, destino]);
  res.status(201).json({ mensaje: 'ruta creada' });
};
```

Asimismo, `frontend/src/api/api.js` define funciones de invocación:

```
export const obtenerRutas = async (token) => {
  const res = await fetch('/api/rutas', { headers: { Authorization: `Bearer
${token}` } });
  return res.json();
};
```

Para un listado completo de rutas y controladores, revisar los archivos en `backend/controllers` y `backend/routes`.

9. Interfaz de Usuario

La UI se compone de:

- Barra de navegación (`Navbar.jsx`) con enlaces condicionales según rol.
- Sidebar (`Sidebar.jsx`) con accesos a secciones de administración.
- Formatos de visualización de mapas (`MapaLeaflet.jsx`) y gráficos (`GraficaComparacion.jsx`, etc.).

Cada pantalla utiliza un diseño responsivo básico; los estilos son mínimos y pueden ampliarse con CSS o librerías.

10. APIs y Servicios Externos

- **API REST propia:** prefijo `/api`. Rutas principales:
 - `/api/auth`: login y registro.
 - `/api/usuarios`, `/api/rutas`, `/api/paradas`, `/api/zonas`, `/api/vehiculos`.
- **Base de datos MySQL:** servicio interno, accedido por `mysql2` o `sequelize`.

No se consumen servicios externos adicionales en la versión actual.

11. Seguridad

- Políticas: roles con autorización en middlewares (`authMiddleware.js`).

- Medidas: validación de entrada, manejo de errores SQL en `dbErrorHandler.js`, uso de HTTPS recomendado en despliegue.
 - Resolución: utilizar logging y revisar registros; en caso de brechas cambiar `JWT_SECRET` y rotar contraseñas.
-

12. Mantenimiento y Actualizaciones

- Ejecutar `npm update` periódicamente.
 - Respalidar la base de datos antes de migrar. El script `migrate-to-mysql.js` soporta reimportar datos de `db.json`.
 - Para migraciones de esquema, añadir nuevo archivo `.sql` y aplicar con `mysql`.
-

13. Resolución de Problemas

- **Servidor no arranca:** verificar variables en `.env`, puertos ocupados.
- **Errores de CORS:** habilitar con `app.use(require('cors')())` en `index.js`.
- **Token expirado:** volver a hacer login.

Frecuentes: problemas de conexión a MySQL, revisar credenciales y que el servicio esté activo.

Este manual puede ser complementado y actualizado conforme el software evoluciona. Se recomienda mantenerlo en la carpeta `docs` bajo control de versiones.